

std::unexpected (my friend)
noexcept override
throw
try
catch

iterator
bad_alloc
reverse
const

<stdexcept> ✓
<exception> ✓
<forward-list> ✓
<list>
<iterator> ✓

GTU CSE 241 / 501 Quiz

Std Code:1188

```
#ifndef container_h
#define container_h
#include <iostream>
#include <forward-list>
#include <iterator>
using namespace std;
#include <string>
#include <exception>
template <class T>
class Exception: public exception{
private:
    T message;
public:
    T& what() noexcept override{
        return message;
    }
    Exception(const T& msg): message(msg){}
};

int findInContainer(forward-list<int> flist, int x){
    forward-list<int>::iterator it;
    for(it = flist.begin(); it != flist.end(); ++it){
        if(flist[it] == x){
            return it;
        }
    }
    throw runtime_error("That value can't found");
}

#endif
```

```
#include "container.h"
```

```
int main() {
```

```
try {
```

```
forward_list<int> flist = {1, 2, 3, 4, 5};
```

```
forward_list<int> olist = {2, 3, 4, 5, 6};
```

```
findInContainer(flist, 2);
```

```
findInContainer(olist, 7);
```

```
if (findInContainer(flist, 2)) {
```

```
cout << "found value 2 on position: " <<
```

```
findInContainer(flist, 2) << endl;
```

```
else {
```

```
cout << "didn't found value 2 in list" << endl;
```

```
if (findInContainer(olist, 7)) {
```

```
cout << "found value 7 on position: " <<
```

```
findInContainer(olist, 7) << endl;
```

```
else {
```

```
cout << "Didn't found value 7 in list" << endl; }
```

```
catch (runtime_error &e) {
```

```
cout << "Error: " << e.what() << endl;
```

```
}
```

```
return 0;
```

```
}
```